# Secure Matrix Invertibility Testing over Fields of Small Order or Characteristics

Seungwoo Han, Jooyoung Lee, Seungmin Park and Mincheol Son

KAIST, Daejeon, South Korea

**Abstract.** Multi-party matrix invertibility testing over finite fields of small order or characteristic is a pivotal operation for thresholdizing Multivariate Quadratic (MQ) signature schemes. However, achieving perfect privacy in a constant number of rounds remains a challenge: existing solutions are not perfectly secure with leakage of certain information or inefficient in terms of computational and communication complexity, in particular, when $p \leq n$, where $p$ and $n$ denote the characteristic of the underlying field and the matrix size, respectively.

To address these limitations, we propose two protocols for perfectly secure multi-party testing of matrix invertibility. The first protocol extends the Cramer-Damgård protocol to fields of small order by employing the field lifting technique. The second protocol is based on a multiparty computation of the Samuelson-Berkowitz algorithm, specifically designed for fields with a small characteristic where $p \leq n$. Both constructions are formalized in the Arithmetic Black-Box (ABB) model with the Shamir's secret sharing scheme.

We show that both protocols achieve perfect privacy with the tradeoff between online and offline rounds. Specifically, the first protocol runs in 7 offline rounds with complexity $O(N \cdot n^4 + n^5)$ and in 3 online rounds with complexity $O(n^3)$, and the second protocol runs in 3 offline rounds with complexity $O(n^3)$ and in 9 online rounds with complexity $O(n^4)$, where $n$ is the matrix size and $N$ is the number of parties.

**Keywords:** Multi-Party Computation · Linear Algebra · Matrix Invertibility Testing

## 1 Introduction

Secure execution of linear algebra algorithms in multi-party setting is a cornerstone of privacy-preserving computation, including Zero-Knowledge Proof via MPC-in-the-Head, private set intersection (PSI), privacy-preserving machine learning (PPML), and threshold Multivariate Quadratic (MQ) signatures. In particular, matrix invertibility testing over finite fields of small order or characteristic is a fundamental building block for thresholdizing MQ signature schemes such as UOV [BCD+24] and MAYO [BCC+23]).

In general, an MPC protocol is designed to achieve perfect privacy, ensuring that no private information is leaked to an adversary during the execution of the protocol. However, while several constant-round threshold MQ signature protocols have been proposed [CS19, CEN25], they often fail to achieve perfect privacy. This information leakage typically originates from the functionality used to test the invertibility of a given matrix $A$ during the computation of its inverse $A^{-1}$. Recently, Maire and Vergnaud proposed a perfectly secure, constant-round protocol for this functionality [MV24]; however, their approach becomes inefficient when $p \leq n$, where $p$ and $n$ denote the characteristic of the underlying field and the size of the matrix, respectively. This constraint is not desirable for typical

E-mail: swhan@kaist.ac.kr (Seungwoo Han), hicalf@kaist.ac.kr (Jooyoung Lee), smpak@kaist.ac.kr (Seungmin Park), encrypted.def@kaist.ac.kr (Mincheol Son)


Date of this document: 2026-03-13.

MQ signature schemes, which rely primarily on computations over a finite field $\mathbb{F}$ with small characteristics such as $p = 2$.

## 1.1 Our contribution

In this paper, we propose two protocols for perfectly secure multi-party testing of matrix invertibility, each of which addresses a specific limitation of existing protocols. The first protocol extends the Cramer-Damgård protocol to fields of small order by employing the field lifting technique [CD01]. The second protocol is based on a multiparty computation of the Samuelson-Berkowitz algorithm [Ber84], specifically designed for fields with a small characteristic where $p \leq n$.

These protocols are described in the *arithmetic black-box* (ABB) model, and its round and communication complexity is analyzed with Shamir's secret sharing. Consequently, each construction is executed in a constant number of rounds with the tradeoff between online and offline rounds, providing perfect privacy. Specifically, the first protocol runs in 7 offline rounds with communication complexity $O(N \cdot n^4 + n^5)$ and in 3 online rounds with communication complexity $O(n^3)$, and the second protocol runs in 3 offline rounds with communication complexity $O(n^3)$ and in 9 online rounds with communication complexity $O(n^4)$, where $n$ is the matrix size and $N$ is the number of parties. Compared to the modified invertibility testing protocol from [MV24] combined with the Schönhage algorithm [Sch93] for small characteristic fields, our first protocol reduces online communication complexity, while both of our proposed protocols minimize the online rounds. To the best of our knowledge, these are the state-of-the-art perfectly secure and efficient MPC protocols for invertibility testing over fields of a small characteristic and can be used as a core building block for the construction of perfectly secure and efficient multivariate threshold signature schemes. The efficiency of our protocols is summarized in Table 1.

**Table 1:** Online and offline round complexities and communication cost of invertibility testing protocols providing perfect privacy over fields with small order or characteristics. For rounds and communication complexities, online and offline costs are distinguished.

| Protocol | Field setting | Rounds (on/off) | Comm. (on/off) |
|---|---|---|---|
| [MV24]+[Sch93] (App. A) | Small char. | 11/2 | $O(n^4)/O(n^3)$ |
| $\mathsf{isInverseCD}(A)$ (Sec. 4) | Small order | **3**/7 | $\boldsymbol{O(n^3)}/O(N \cdot n^4 + n^5)$ |
| $\mathsf{isInverseSB}(A)$ (Sec. 5) | Small char. | **9**/3 | $O(n^4)/O(n^3)$ |

## 1.2 Notation

We use standard notation from secure multi-party computation and finite field linear algebra. Let $\mathbb{F}_q$ denote the finite field of size $q$, and let $\mathbb{F}_{q^k}$ denote its degree-$k$ extension field. For $n \in \mathbb{N}$, we write $\mathcal{M}_n(\mathbb{F}_q) = \mathbb{F}_q^{n \times n}$ for the set of $n \times n$ matrices over $\mathbb{F}_q$, and $\mathrm{GL}_n(\mathbb{F}_q) \subseteq \mathcal{M}_n(\mathbb{F}_q)$ for the subset of invertible matrices.

We write $r \xleftarrow{\$} S$ to denote sampling $r$ uniformly at random from a finite set $S$. For a matrix $A \in \mathcal{M}_n(\mathbb{F}_q)$, $\chi_A(x) = \det(xI_n - A)$ denotes its characteristic polynomial.

Throughout the paper, we describe protocols in the Arithmetic Black-Box (ABB) model. A value $x \in \mathbb{F}$ stored in the ABB is denoted by a sharing $[x]$. When needed, we use $[x]^{(i)}$ to denote the local share held by party $P_i$. For matrix- and vector-valued data, the same bracket notation is used entry-wise, e.g., $[A] \in \mathbb{F}^{n \times n}$ denotes an entry-wise sharing of $A \in \mathbb{F}^{n \times n}$. We use the standard ABB interfaces $\mathsf{dist}(\cdot)$, $\mathsf{rand}(\cdot)$, and $\mathsf{open}(\cdot)$ as defined in Section 2.1.

In Section 4, computations are performed over $\mathbb{F}_{q^{n+1}}$ by *lifting* inputs from $\mathbb{F}_q$. We denote by $A^{\mathsf{lift}}$ the natural embedding of $A \in \mathcal{M}_n(\mathbb{F}_q)$ into $\mathcal{M}_n(\mathbb{F}_{q^{n+1}})$ obtained by viewing each entry of $A$ as an element of the extension field.

For complexity measures, we report the number of rounds and the communication cost. We split communication into *offline* (input-independent) and *online* (input-dependent) components as described in Section 1.3. Throughout the paper, we measure communication cost by the total number of communicated field elements (over the relevant field for the protocol instantiation).

We summarize the notations in Table 2.

**Table 2:** Notations used in this paper.

| Notation | Meaning |
|---|---|
| $N$ | Number of parties. |
| $p$ | Field characteristic. |
| $q$ | Field order. |
| $\mathbb{F}_q$ | Finite field with $q$ elements. |
| $\mathbb{F}_{q^k}$ | Degree-$k$ extension field of $\mathbb{F}_q$. |
| $n$ | Matrix dimension. |
| $\mathcal{M}_n(\mathbb{F})$ | Set of $n \times n$ matrices over $\mathbb{F}$. |
| $\mathrm{GL}_n(\mathbb{F})$ | Set of invertible $n \times n$ matrices over $\mathbb{F}$. |
| $A$ | Input matrix, typically $A \in \mathcal{M}_n(\mathbb{F}_q)$. |
| $I_n$ | $n \times n$ identity matrix. |
| $\chi_A(x)$ | Characteristic polynomial $\det(xI_n - A)$. |
| $\xleftarrow{\$}$ | Uniform sampling, e.g., $r \xleftarrow{\$} S$. |
| $[x]$ | Sharing of $x$ (secret-shared value). |
| $[x]^{(i)}$ | Local share of $[x]$ held by party $P_i$. |
| $\mathsf{dist}(P_i, x)$ | Input distribution of $x$ by $P_i$. |
| $\mathsf{rand}(\mathbb{F})$ | Generation of a random shared element in $\mathbb{F}$. |
| $\mathsf{open}([x])$ | Opening (reconstruction) of a shared value. |
| $A^{\mathsf{lift}}$ | Natural embedding of $A$ into an extension field matrix space. |

## 1.3 Security Assumption and Model

We assume that an underlying Multi-Party Computation (MPC) framework provides an *arithmetic black-box* (ABB) functionality, supporting linear operations, multiplication, input distribution, random element generation, and value opening with secure implementations. While our primary focus and evaluation target the semi-honest setting, we describe the algorithm within the general ABB model. Consequently, the construction can be instantiated in the malicious settings by using an MPC framework secure against malicious adversaries, such as SPDZ [DPSZ12] or MPC protocols based on replicated secret sharing [AFL+16]. We further discuss this aspect in Section 2.2.

For the evaluation of MPC protocols, we mainly consider round complexity and communication complexity. We divide the communication cost into two phases: the *offline phase*, which consists of input-independent communication, and the *online phase*, which consists of input-dependent communication.

In practical deployments, the offline phase can be executed during idle time or replaced by a trusted-dealer model in which correlated randomness is distributed in advance. Therefore, we prioritize reducing the online cost, as it typically constitutes the dominant component of end-to-end latency.

## 1.4 Related Work

In plain settings, matrix invertibility testing is typically performed using Gaussian elimination. However, its direct adaptation to MPC presents significant challenges. Specifically, its high round complexity and reliance on conditional pivoting operations make it inefficient for secure computation. Instead of adapting these sequential algorithms, prior works have proposed various constant round protocols [BIB89, CD01, CKP07, MW08, MV24]. We will survey these prior works and highlighting the limitations that our work aims to address in this section.

**The Protocol of Bar-Ilan-Beaver.** Bar-Ilan and Beaver firstly proposed various constant-round multi-party secure protocols such as computing inverse of group element, unbounded matrix multiplication, and generating invertible matrices having full rank [BIB89]. For a given group $(G, \cdot)$ and group element $[g]$, the basic idea of computing $[g^{-1}]$ is masking $[g]$ with random group element $[r]$ so that each party reveals $[r \cdot g]$ and compute its inverse locally. Then each party can compute an inverse $[g^{-1}] = (r \cdot g)^{-1} \cdot [r]$ securely. We can utilize above idea for secure matrix invertibility testing for $A \in \mathcal{M}_n(\mathbb{F}_q)$ using preprocessed random $R \in \mathrm{GL}_n(\mathbb{F}_q)$, by computing determinant of inverse of masked matrix $(A \cdot R)^{-1}$. As it masks only one side of given matrix, it is so-called one-sided masking protocol. This protocol used to construct the first threshold MQ signature [CS19].

After that, Celi, Escudero and Niot showed that one-sided masking protocol can potentially reveals column space of matrix $A$ thus increasing guessing probability of oil space [CEN25]. Instead, they proposed two-sided masking protocol with two random invertible matrices $R, S \in \mathrm{GL}_n(\mathbb{F}_q)$ so that parties compute determinant of $(R \cdot A \cdot S)^{-1}$. Despite this two-sided masking protocol prevents leakage of $A$'s column space, it still leaks rank information of $A$ when $A$ is singular.

**The Protocol of Cramer-Damgård.** Cramer and Damgård proposed a constant-round protocol for several linear algebra tasks, including determinant computation, rank evaluation, and solving linear systems over finite fields [CD01]. Their protocol of invertibility testing exploits properties of the characteristic polynomial and was intended to provide strong privacy guarantees.

For a given matrix $A \in \mathcal{M}_n(\mathbb{F}_q)$, let $\chi_A(x)$ denote the characteristic polynomial of $A$, defined by:

$$\chi_A(x) = \det(xI_n - A).$$

Since $\chi_A(x)$ has degree at most $n$, it is uniquely determined by its values at $n+1$ distinct field elements. Moreover, the determinant of $A$ is encoded in the constant term:

$$\chi_A(0) = \det(-A) = (-1)^n \det(A).$$

Hence, computing $\det(A)$ reduces to evaluating $\chi_A(0)$.

The protocol fixes $n+1$ public distinct points $z_1, z_2, \ldots, z_{n+1} \in \mathbb{F}$. For any $z \in \mathbb{F}$, the matrix $(zI_n - A)$ is invertible if and only if $z$ is not an eigenvalue of $A$. Since $A$ has at most $n$ eigenvalues, a uniformly random $z$ yields an invertible $(zI_n - A)$ with probability at least $1 - n/q$. When $q$ is sufficiently large, the matrices $M_i = z_i I_n - A$ are therefore invertible with high probability.

The parties jointly compute shared values $[y_i] = [\det(M_i)]$. This step requires a shared random invertible matrix $[R]$ together with a sharing of its determinant $[\det(R)]$. Given the pairs $\{(z_i, [y_i])\}_{i \in [n+1]}$, the parties reconstruct a sharing of $[\chi_A(0)]$ via Lagrange interpolation:

$$[\chi_A(0)] = \sum_{i \in [n+1]} \lambda_i [y_i], \tag{1}$$

where $\lambda_i$ are the public Lagrange coefficients.

This protocol enables computation of $\det(A)$ even when $A$ may be singular, by evaluating determinants of $n+1$ matrices in parallel. Nevertheless, it exhibits several limitations. First, each $M_i$ is singular with probability at most $n/q$. The original analysis assumes $q$ sufficiently large so that $n/q$ can be neglected, but this term can be non-negligible over small field order. Second, the protocol does not achieve perfect privacy in general. During execution, the rank of each $M_i$ becomes public; in particular, observing that $M_i$ has full rank implies that $z_i$ is not an eigenvalue of $A$. When $q$ is small, such information can be exploited to infer properties of $A$. Finally, the generation of $[R]$ and $[\det(R)]$ in the original construction relies on LU decomposition. Concretely, the parties sample a lower-triangular matrix $[L]$ and an upper-triangular matrix $[U]$, set $[R] = [L] \cdot [U]$, and compute $[\det(R)] = [\det(L)] \cdot [\det(U)]$, where $\det(L)$ and $\det(U)$ are obtained as products of diagonal entries. However, the induced distribution of $[R]$ is not uniform over $\mathrm{GL}_n(\mathbb{F}_q)$, which may further degrade privacy.

**The Protocol of Maire-Vergnaud.** Maire and Vergnaud [MV24] proposed protocols for secure linear algebra, utilizing Leverrier's Lemma to compute the characteristic polynomial and determinant of a shared matrix. This protocol provides a deterministic protocol for invertibility testing, which serves as a foundational technique in their work.

**Lemma 1** (Leverrier)**.** *Let $A \in \mathcal{M}_n(\mathbb{F})$ be a matrix and let $\chi_A(X) = \det(xI_n - A) = x^n + d_1 x^{n-1} + \cdots + d_n$ be its characteristic polynomial. Let $t_k = \mathrm{Tr}(A^k)$ denote the power sums of the eigenvalues of $A$. The coefficients $d_k$ satisfy the following relation:*

$$\begin{pmatrix} 1 & 0 & \cdots & 0 \\ t_1 & 2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ t_{n-1} & t_{n-2} & \cdots & n \end{pmatrix} \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_n \end{pmatrix} = - \begin{pmatrix} t_1 \\ t_2 \\ \vdots \\ t_n \end{pmatrix}$$

If the field characteristic $p$ is greater than $n$, the matrix on the left is invertible, and one can recover the coefficients $d_k$ (and thus the determinant $\det(A) = (-1)^n d_n$) by computing the traces of the powers $A, A^2, \ldots, A^n$.

Based on this lemma, testing the invertibility of a shared matrix $[A]$ reduces to verifying whether its determinant is non-zero. Given the identity $\det(A) = (-1)^n d_n$, this is equivalent to testing if the constant term $d_n$ of the characteristic polynomial is non-vanishing. The protocol initiates by computing the shares of the matrix powers $[A], [A^2], \ldots, [A^n]$ via Algorithm 6. This algorithm employs a baby-step giant-step strategy—computing powers up to $A^{\lceil \sqrt{n} \rceil}$ and subsequently powers of that result—to optimize the round complexity. Upon obtaining these powers, the parties locally compute the traces $\mathrm{Tr}([A^k])$. Let $[M]$ denote the shared lower-triangular matrix constructed from these traces as defined in Leverrier's Lemma. To solve the system for $[d_n]$, the parties perform a secure matrix inversion: they generate a shared random invertible matrix $[R]$ and a random non-zero element $[r]$ (using Algorithm 2), compute the masked matrix $[Z] = [R] \cdot [M]$, and open $[Z]$. The inverse is then recovered as $[M^{-1}] = Z^{-1} \cdot [R]$, allowing the derivation of $[d_n]$. Finally, the parties perform the invertibility test by computing $[z] = [r] \cdot [d_n]$ and open $[z]$; the matrix $[A]$ is invertible if and only if $z \neq 0$.

# 2 Preliminaries

## 2.1 Arithmetic Black-Box Model

We describe our protocol in the *Arithmetic Black-Box (ABB)* model, a standard abstraction for secure multi-party computation (MPC). In this model, an ideal functionality $\mathcal{F}_{\mathrm{ABB}}$ enables a set of parties $\mathcal{P} = \{P_1, \ldots, P_N\}$ to store elements as secret shared variables and

to perform arithmetic operations over these shared values. For $x \in \mathbb{F}$, its representation within the ABB is denoted by $[x]$. The concrete sharing protocol underlying $[x]$ (e.g., Shamir's secret shares or additive shares with MACs) is intentionally abstracted and does not appear in the protocol specification.

The functionality provides the following interfaces:

- **Linear operations** ($[z] \leftarrow [x] + [y]$, $[z] \leftarrow a \cdot [x] + b$): Given shared values $[x], [y]$ and public constants $a, b \in \mathbb{F}$, the functionality outputs $[z]$ such that $z = x + y$ or $z = ax + b$. For most linear secret sharing schemes (LSSS), these computations are local and do not require interaction.
- **Secure multiplication** ($[z] \leftarrow [x] \cdot [y]$): Given shared values $[x]$ and $[y]$, the functionality outputs $[z]$ such that $z = x \cdot y$. In contrast to linear operations, this procedure typically requires interaction and may consume preprocessed correlated randomness (e.g., Beaver triples) depending on the underlying instantiation.
- **Input distribution** ($[x] \leftarrow \mathsf{dist}(P_i, x)$): A party $P_i$ submits a private input $x \in \mathbb{F}$. The functionality stores $x$ and distributes a sharing $[x]$ to the parties.
- **Random element generation** ($[r] \leftarrow \mathsf{rand}(\mathbb{F})$): The functionality samples a uniformly random element $r \xleftarrow{\$} \mathbb{F}$ and distributes its sharing $[r]$. The value $r$ remains information-theoretically hidden from the parties until it is opened.
- **Value opening** ($x \leftarrow \mathsf{open}([x])$): Given a shared value $[x]$, the functionality reconstructs and reveals the corresponding value $x$ to all parties.

Although these interfaces are specified for scalar values over $\mathbb{F}$, we adopt the same notation for vectors and matrices whenever this causes no ambiguity. For instance, $[R] \leftarrow \mathsf{rand}(\mathbb{F}^{m \times m})$ denotes the generation of a shared $m \times m$ matrix whose entries are sampled independently and uniformly from $\mathbb{F}$. Likewise, linear operations and openings apply coordinate-wise to such structured values.

For vectors and matrices, the interfaces for linear operations, $\mathsf{dist}$, $\mathsf{rand}$, and $\mathsf{open}$ extend directly in a component-wise fashion. Moreover, secure matrix multiplication can be implemented generically by composing the corresponding scalar multiplications.

Nevertheless, this straightforward decomposition is not necessarily performance-optimal. Depending on the underlying MPC instantiation, matrix-level primitives and other specialized techniques may reduce interaction and communication relative to a purely scalar realization. We defer a detailed discussion of such optimized implementations and their performance implications to Section 2.2.

## 2.2 MPC Protocols

Although our protocol is well defined on a any MPC protocols support the interfaces defined in Section 2.1, we specify MPC protocols to provide communication complexities and round numbers.

We delineate the core cryptographic primitives utilized throughout this work, formally defining the network topology and the operational semantics for Shamir's Secret Sharing. Special emphasis is placed on the secure inner product protocol, which acts as a vectorized generalization of standard multiplication to support the linear algebraic constructions presented later. The derived communication complexities and round counts for each sub-protocol are summarized in Table 3.

**Communication Model.** The protocol operates within a network of $N$ parties $\mathcal{P} = \{P_1, \ldots, P_N\}$ connected via a fully connected topology. We assume the existence of $\binom{N}{2}$ secure point-to-point channels, where every distinct pair of parties shares a dedicated link ensuring both perfect confidentiality and authentication. We do not assume the availability of a physical broadcast channel; consequently, all global data dissemination is achieved through multiple unicast transmissions over the established point-to-point links.

This framework is specifically instantiated to facilitate the asymptotic analysis of communication complexity and round efficiency under the assumption of an honest-majority ($t < N/2$) facing an semi-honest adversary. By isolating the fundamental transmission cost inherent to the network topology, we establish a baseline for information-theoretic performance. It is important to note that this model is modular; adapting protocols for stronger adversarial settings (e.g., malicious behavior) or weaker trust assumptions (e.g., dishonest majority) would necessitate augmenting this foundation with additional primitives, such as broadcast channels or verifiable consistent transfer mechanisms, which would strictly increase the resultant complexity overhead.

**Communication Cost.** To evaluate the efficiency of the protocols independent of the network size, we define the communication cost as traffic load per secure point-to-point channel. Let $\mathcal{V}_{\text{total}}$ denote the aggregate volume of data (measured in field elements) transmitted across the entire network during the execution of a protocol. Given a fully connected topology with $N$ parties, the total number of distinct directed links is $N(N-1)$. The communication cost $\mathcal{C}$ is computed by normalizing the total volume by the number of links:

$$\mathcal{C} = \frac{\mathcal{V}_{\text{total}}}{N(N-1)}$$

Under this metric, an "all-to-all" exchange where every party transmits a single field element to every other party yields a normalized communication cost of exactly 1.

**Shamir's Secret Sharing.** To share a secret $s$ in Shamir's scheme, a random polynomial $f(x)$ of degree $t$ is generated such that $f(0) = s$. Each party $i$ receives a share $[x]^{(i)} = f(i)$, and the secret is reconstructed using Lagrange interpolation with any subset of at least $t+1$ shares. The scheme conceals $s$ as the constant term of a degree $t$ polynomial $f(x)$ over a finite field. Unique points $(i, f(i))$ are distributed to parties, requiring a threshold of $t$ points to define the polynomial and recover $s$. The basic algorithms are listed in Figure 1.

**Secure Multiplications.** The secure inner product protocol generalizes to scalar multiplication (where $n = 1$) and matrix multiplication. Because the product of $p \times q$ and $q \times r$ matrices decomposes into $pr$ independent inner products of dimension $q$, parallel execution preserves constant round complexity. Consequently, we have a multiplication protocol whose communication complexity scales linearly with the output size $pr$ relative to a single inner product at worst.

# 3 Building Blocks

In this section, we introduce basic building blocks for our protocols. Overall complexities are provided in Table 4.

## 3.1 Generating Nonzero Elements

We present a constant-round protocol to generate $m$ nonzero elements in Algorithm 1.

The algorithm proceeds by generating $c$ pairs of random elements $w_j$ and computing their products $[w_j] = [u_j] \cdot [v_j]$. The product $W_j$ is opened. If $w_j$ is nonzero, it implies that both $u_j$ and $v_j$ are nonzero. The parties can then take $u_j$ as a random nonzero element and compute its inverse as $[u_j^{-1}] = [v_j] \cdot w_j^{-1}$.

The number of candidates $c$ is determined based on the probability that a random element is nonzero. Let $\alpha = (1 - q^{-1})^2$ be the probability that two independent uniformly random elements in $\mathbb{F}_q$ are both nonzero. By setting $c = \lceil (m + \delta\sqrt{m} + \delta^2)/\alpha \rceil / m$ for a constant $\delta > 0$, the Chernoff bound guarantees that the parties obtain at least $m$ nonzero elements in the first loop with high probability. Table 5 presents the ratios $c$ for selected values of $m$, $\alpha$, and $P_{\text{fail}}$. Even if the algorithm fails to generate $m$ nonzero elements

**Algorithm:** Addition

**Input** : Shares $[x]$ and $[y]$
**Output** : Share $[x+y]$

1 The parties locally compute their respective shares $[x+y]^{(i)} = [x]^{(i)} + [y]^{(i)}$ for $i \in [N]$.

**Algorithm:** Multiplication by public constant

**Input** : Share $[x]$ and public value $c$
**Output** : Share $[cx]$

1 The parties locally compute their respective shares $[cx]^{(i)} = c[x]^{(i)}$ for $i \in [N]$.

**Algorithm:** Input distribution

**Input** : A private input $x$ and a party $j$
**Output** : Share $[x]$

1 Party $j$ generates random polynomial $f(X)$ of degree at most $t$ with $f(0) = x$.
2 Party $j$ sends $[x]^{(i)} = f(i)$ to party $i$.
3 Party $j$ sets its share $[x]^{(j)} = f(j)$.

**Algorithm:** Random Element Generation [CDI05]

**Input** : None
**Output** : Share $[r]$ where $r \xleftarrow{\$} \mathbb{F}_q$

1 Let $\mathcal{S} = \{S \in \{P_1, \ldots, P_N\} : |S| = N - t\}$ be the collection of all subsets of $N - t$ parties. For each set $S \in \mathcal{S}$, $N - t$ parties shares a secret key $k_S$.
2 Each set $S$ is associated with a (public) polynomial $f_S(X)$ of degree at most $t$ such that $f_S(0) = 1$ and $f_S(j) = 0$ for all $P_j \notin S$.
3 For each subset $S$, each party $P_j \in S$ computes $r_S = \mathsf{PRF}(k_S, \mathsf{nonce})$ where $\mathsf{nonce}$ is a public nonce.
4 For each party $i$, let $\mathcal{I}_i = \{S \in \mathcal{S} : P_i \in S\}$ denote the family of sets for which $P_i$ holds the key $k_S$. Then $P_i$ locally computes $[r]^{(i)} = \sum_{S \in \mathcal{I}_i} r_S f_S(i)$.

**Algorithm:** Value Opening

**Input** : Share $[x]$
**Output** : Public $x$

1 The parties send their shares $[x]^{(i)}$ to $P_{i+1}, \ldots, P_{i+t}$ (indices $\mod N$) for $i \in [N]$.
2 The parties locally reconstruct $x$ using the $t+1$ available shares.

**Algorithm:** Secure Multiplication [BGW88]

**Input** : Shares $[x]$ and $[y]$
**Output** : Share $[x \cdot y]$

1 Let $\{\lambda_i\}_{i=1}^N$ be Lagrange coefficients, i.e., $\lambda_i = \prod_{j \neq i} \frac{0-j}{i-j} \in \mathbb{F}_q$
2 The parties re-share their local sum $z_i = \sum_{k \in [n]} [x]^{(i)} \cdot [y]^{(i)}$ by invoking $\mathsf{dist}(P_i, z_i)$ for $i \in [N]$.
3 The parties locally compute their respective shares $[x \cdot y]^{(i)} = \sum_{j \in [N]} \lambda_j [z_j]^{(i)}$ for $i \in [N]$.

**Algorithm:** Secure Inner Product [BGW88]

**Input** : Shared vectors $[\mathbf{x}]$ and $[\mathbf{y}]$ of length $n$
**Output** : Share $[\mathbf{x} \cdot \mathbf{y}]$

1 Let $\{\lambda_i\}_{i=1}^N$ be Lagrange coefficients, i.e., $\lambda_i = \prod_{j \neq i} \frac{0-j}{i-j} \in \mathbb{F}_q$
2 The parties re-share their local sum $z_i = \sum_{k \in [n]} [\mathbf{x}_k]^{(i)} \cdot [\mathbf{y}_k]^{(i)}$ by invoking $\mathsf{dist}(P_i, z_i)$ for $i \in [N]$.
3 The parties locally compute their respective shares $[\mathbf{x} \cdot \mathbf{y}]^{(i)} = \sum_{j \in [N]} \lambda_j [z_j]^{(i)}$ for $i \in [N]$.

**Figure 1:** Basic MPC protocols for Shamir's sharing.

**Table 3:** Online round complexity and communication cost (field elements per channel)

| Operation | Rounds | Comm. |
|---|---|---|
| Addition & Multiplication by Public Value | 0 | 0 |
| Secure Multiplication | 1 | 1 |
| Input Distribution | 1 | $1^*$ |
| Random Element | $0^\dagger$ | $0^\dagger$ |
| Opening | 1 | 1 |
| Inner Product (dimension $n$) | 1 | 1 |
| Matrix Multiplication ($p \times q$ and $q \times r$) | 1 | $pr$ |

$^*$ Transmission load is asymmetric, borne exclusively by the dealer.
$^\dagger$ Zero-communication generation requires Pseudorandom Secret Sharing (PRSS); scalability is limited by the combinatorial offline storage requirement and local computation.

**Table 4:** Online and offline round complexities and communication cost of building block protocols. For rounds and communication complexities, online and offline costs are distinguished. Constants $c_1$ and $c_2$ are determined in Section 3.1 and Section 3.3, respectively.

| Protocol | Rounds (on/off) | Comm. (on/off) |
|---|---|---|
| $\mathsf{GenNonzeroElems}(\mathbb{F}, m)$ (Sec. 3.1) | 2/0 | $2c_1m/0$ |
| $\mathsf{GenInvMats}(\mathbb{F}, m, n)$ (Sec. 3.2) | 2/0 | $2c_2mn^2/0$ |
| $\mathsf{MultScalars}(\mathbb{F}, a_1, \ldots, a_t)$ (Sec. 3.3) | 3/2 | $3t - 1/2c_1t$ |
| $\mathsf{MultMats}(\mathbb{F}, [A_1], \ldots, [A_t])$ (Sec. 3.4) | 3/2 | $(3t-1)n^2/2c_2tn^2$ |
| $\mathsf{GenInvMatDet}(\mathbb{F})$ (Sec. 3.5) | 4/2 | $(4N-1)(n^2+1)/(2c_1+2c_2n^2)N$ |
| $\mathsf{SecPower}(\mathbb{F}, A, m)$ (Sec. 3.6) | 3/2 | $(3m-1)n^2/2c_2mn^2$ |

in the first loop, the parties may perform the remaining elements in the next loop with overwhelming probability.

**Algorithm 1:** $\mathsf{GenNonzeroElems}(\mathbb{F}, m)$ - Generate $m$ nonzero elements

**Input :** Integer $m$
**Output :** $m$ pairs of nonzero elements and its inverses $([r_k], [r_k^{-1}])$ where $r_k \in \mathbb{F} \setminus \{0\}$

Set constants $\alpha \leftarrow (1 - q^{-1})^2$ and $c_1 \leftarrow \lceil (m + \delta\sqrt{m} + \delta^2)/\alpha \rceil / m$.
Initialize list $L \leftarrow \emptyset$.
**while** $|L| < m$ **do**
    The parties locally generate $c_1 m$ pairs of random elements $([u_j], [v_j])$ where $u_j \leftarrow \mathsf{rand}(\mathbb{F})$ and $v_j \leftarrow \mathsf{rand}(\mathbb{F})$ for $j \in \{1, \ldots, c_1\}$.
    The parties compute and open $w_j \leftarrow \mathsf{open}([u_j] \cdot [v_j])$ for $j \in \{1, \ldots, c_1\}$.
    **for** $j = 1$ **to** $c_1 m$ **do**
        **if** $w_j$ *is invertible* **then**
            Compute $[r] \leftarrow [u_j]$ and $[r^{-1}] \leftarrow [v_j] \cdot w_j^{-1}$.
            Append $([r], [r^{-1}])$ to $L$.
**return** first $m$ pairs in $L$

**Cost.** The protocol runs in a loop that terminates with overwhelming probability in the first iteration. Within the loop, the operations consist of generating random element (local), followed by multiplication and opening (2 rounds). Assuming that the protocol terminates in the first iteration, the protocol runs in 2 rounds and requires $c_1 m$ multiplications and $c_1 m$ openings of the matrices. Consequently, the total communication cost amounts to $2c_1 m$ field elements.

**Table 5:** Required candidate generation ratio $c$ to achieve $m$ invertible matrices, parameterized by $P_{\text{fail}}$ and $1 - \alpha$.

| | | $c$ | | |
|---|---|---|---|---|
| $1-\alpha$ | $P_{\text{fail}}$ | $m = 50$ | $m = 100$ | $m = 200$ |
| $2^{-1}$ | $2^{-40}$ | 6.34 | 4.60 | 3.61 |
| | $2^{-64}$ | 8.22 | 5.66 | 4.22 |
| | $2^{-128}$ | 12.88 | 8.22 | 5.66 |
| $2^{-4}$ | $2^{-40}$ | 3.38 | 2.46 | 1.93 |
| | $2^{-64}$ | 4.40 | 3.02 | 2.25 |
| | $2^{-128}$ | 6.88 | 4.39 | 3.02 |
| $2^{-8}$ | $2^{-40}$ | 3.18 | 2.31 | 1.81 |
| | $2^{-64}$ | 4.14 | 2.85 | 2.12 |
| | $2^{-128}$ | 6.46 | 4.13 | 2.85 |

**Lemma 2.** *Let $X$ be the number of nonzero elements found in a single batch of size $cm$. If the batch size is chosen as $cm = \lceil (m + \delta\sqrt{m} + \delta^2)/\alpha \rceil$ where $\alpha = (1 - q^{-1})^2$ is the probability that two random elements are all nonzero, then the probability of finding fewer than $m$ nonzero elements is bounded by:*

$$P[X < m] \leq e^{-\delta^2/2}.$$

*Proof.* Let $X = \sum_{j \in [cm]} X_j$ be the sum of independent Bernoulli random variables where $X_j = 1$ with probability $\alpha$. The expected number of successes is $\mu = \mathbb{E}[X] = cm\alpha$. From our choice of $c$, we have the lower bound:

$$\mu \geq m + \delta\sqrt{m} + \delta^2.$$

We analyze the failure probability $P[X < m]$ using the Chernoff bound for the lower tail. For any deviation $\Delta > 0$ such that $\mu = m + \Delta$, the bound states:

$$P[X \leq m] \leq \exp\left(-\frac{(\mu - m)^2}{2\mu}\right) = \exp\left(-\frac{\Delta^2}{2(m+\Delta)}\right).$$

From our choice of $c$, we have $\Delta \geq \delta\sqrt{m} + \delta^2$. To prove the lemma, it suffices to show that the exponent term satisfies:

$$\frac{\Delta^2}{2(m+\Delta)} \geq \frac{\delta^2}{2} \iff \Delta^2 \geq \delta^2(m+\Delta).$$

We substitute $\Delta = \delta\sqrt{m} + \delta^2$ into the expression $\Delta^2 - \delta^2\Delta - \delta^2 m$:

$$\begin{aligned}\Delta^2 - \delta^2\Delta - \delta^2 m &= (\delta\sqrt{m} + \delta^2)^2 - \delta^2(\delta\sqrt{m} + \delta^2) - \delta^2 m \\ &= (\delta^2 m + 2\delta^3\sqrt{m} + \delta^4) - (\delta^3\sqrt{m} + \delta^4) - \delta^2 m \\ &= \delta^3\sqrt{m}.\end{aligned}$$

Since $\delta > 0$ and $m \geq 1$, we have $\delta^3\sqrt{m} \geq 0$. This confirms that $\frac{\Delta^2}{2(m+\Delta)} \geq \frac{\delta^2}{2}$, and therefore:

$$P[X < m] \leq e^{-\delta^2/2}.$$

Thus, the failure probability is strictly bounded by a term dependent only on $\delta$. □

## 3.2 Generating Invertible Matrices

We present a constant-round protocol to generate $m$ random invertible matrices in Algorithm 2. The protocol is extending Algorithm 1 by changing $u_j, v_j$ into $U_j, V_j$, which are the random matrices expected to be invertible, and set $[W_j] = [U_j] \cdot [V_j]$.

Our construction follows the blueprint of Maire and Vergnaud [MV24], but we introduce a modified batching parameter $c_2$. As in Algorithm 1, this modification is necessary to correct an oversight in the original probabilistic analysis and ensures the protocol terminates in constant expected rounds.

The probability of obtaining at least $m$ invertible matrices with high probability in the first loop is also bounded from Lemma 2, by changing

$$\alpha = \left(\frac{|\mathrm{GL}_n(\mathbb{F}_q)|}{|\mathcal{M}_n(\mathbb{F}_q)|}\right)^2 = \left(\prod_{j=1}^{n}(1 - q^{-j})\right)^2$$

when the field is $\mathbb{F}_q$.

**Algorithm 2:** $\mathsf{GenInvMats}(\mathbb{F}, m, n)$ - Generate $m$ Invertible Matrices of size $n$

**Input** : Integer $m, n$
**Output** : $m$ pairs of shared matrices $([R_k], [R_k^{-1}])$ where $R_k \in \mathrm{GL}_n(\mathbb{F})$

Set constants $\alpha \leftarrow (|\mathrm{GL}_n(\mathbb{F})|/|\mathcal{M}_n(\mathbb{F})|)^2$ and $c_2 \leftarrow \lceil (m + \delta\sqrt{m} + \delta^2)/\alpha \rceil / m$.
Initialize list $L \leftarrow \emptyset$.
**while** $|L| < m$ **do**
  The parties locally generate $c_2 m$ pairs of random matrices $([U_j], [V_j])$ where $U_j \leftarrow \mathsf{rand}(\mathcal{M}_n(\mathbb{F}))$ and $V_j \leftarrow \mathsf{rand}(\mathcal{M}_n(\mathbb{F}))$ for $j \in \{1, \ldots, c_2\}$.
  The parties compute and open $W_j \leftarrow \mathsf{open}([U_j] \cdot [V_j])$ for $j \in \{1, \ldots, c_2\}$.
  **for** $j = 1$ **to** $c_2 m$ **do**
    **if** $W_j$ *is invertible* **then**
      Compute $[R] \leftarrow [U_j]$ and $[R^{-1}] \leftarrow [V_j] \cdot W_j^{-1}$.
      Append $([R], [R^{-1}])$ to $L$.
**return** first $m$ pairs in $L$

**Cost.** The protocol runs in a loop that terminates with overwhelming probability in the first iteration. Within the loop, the operations consist of generating random shared matrices (local), followed by matrix multiplication and opening (2 rounds). Assuming that the protocol terminates in the first iteration and enough number of random invertible matrices are given in prior, the protocol runs in 2 rounds and requires $c_2 m$ matrix multiplications and $c_2 m$ openings of the matrices. Consequently, the total communication cost amounts to $2c_2 mn^2$ field elements.

## 3.3 Multiplying Nonzero Scalar Elements

Algorithm 3 outlines the procedure for computing the product of $t$ shared nonzero scalar elements $[a_1], \ldots, [a_t]$. In the first phase, the parties generate $t$ random nonzero values $r_1, \ldots, r_t$ along with their inverses $r_1^{-1}, \ldots, r_t^{-1}$; this step requires 2 rounds and can be performed offline. Subsequently, the parties compute and open the values $c_i = r_i \cdot a_i \cdot r_{i+1}^{-1}$ for each $i \in \{1, \ldots, t-1\}$, and $c_t = r_t \cdot a_t$. Finally, the sharing of the product is computed as $[b] = [r_1^{-1}] \cdot (c_1 \cdot \cdots \cdot c_t)$.

**Algorithm 3:** $\mathsf{MultScalars}(\mathbb{F}, a_1, \ldots, a_t)$ - Multiply $t$ Nonzero Scalar Elements

**Input** : $[a_1], \cdots, [a_t]$, where $a_i \in \mathbb{F} \setminus \{0\}$ for all $i \in [t]$
**Output**: $[b] = [a_1 a_2 \cdots a_t]$

1 The parties generate $t$ random pairs $([r_i], [r_i^{-1}])_{i \in [t]} \leftarrow \mathsf{GenNonzeroElems}(\mathbb{F}, t)$.
2 **for** $i = 1$ **to** $t$ **do**
3 The parties compute and open $c_i \leftarrow \begin{cases} \mathsf{open}([r_i] \cdot [a_i] \cdot [r_{i+1}^{-1}]) & \text{if } i < t \\ \mathsf{open}([r_t] \cdot [a_t]) & \text{if } i = t. \end{cases}$
4 The parties compute $[b] \leftarrow [r_1^{-1}] \cdot (c_1 \cdot \cdots \cdot c_t)$.
5 **return** $[b]$

**Cost.** The generation of random pairs $([r_i], [r_i^{-1}])$ is independent of the input and can therefore be precomputed in an offline phase. Offline complexity is 2 rounds and $2c_1 t$ field elements, where $c_1$ is a constant defined in Section 3.1.

Online round is of 3 rounds, derived entirely from the operations in Line 3, which involve two multiplications and one opening. Algorithm 3 entails $2t-1$ multiplications and $t$ openings. Consequently, the total communication cost amounts to $3t-1$ field elements.

## 3.4 Multiplying Invertible Matrices

We detail the protocol for calculating the product of $t$ shared invertible matrices, denoted as $[A_1], \ldots, [A_t]$, in Algorithm 4. The process begins with the generation of $t$ random invertible matrices $R_1, \ldots, R_t$ and their corresponding inverses $R_1^{-1}, \ldots, R_t^{-1}$ using Algorithm 2. This initialization spans 2 rounds and is suitable for offline execution. Following this, parties compute and reveal $C_i = R_i \cdot A_i \cdot R_{i+1}^{-1}$ for indices $i$ from 1 to $t-1$, while computing $C_t = R_t \cdot A_t$ for the final term. The shared result $[B]$ is then reconstructed via the formula $[B] = [R_1^{-1}] \cdot (C_1 \cdot \cdots \cdot C_t)$.

**Algorithm 4:** $\mathsf{MultMats}(\mathbb{F}, [A_1], \ldots, [A_t])$ - Multiply $t$ Invertible Matrices

**Input** : $[A_1], \ldots, [A_t]$, where $A_i \in \mathrm{GL}_n(\mathbb{F})$ for all $i \in [t]$
**Output** : $[B] = [A_1 A_2 \cdots A_t]$

1 The parties generate $t$ random pairs $([R_i], [R_i^{-1}])_{i\in[t]} \leftarrow \mathsf{GenInvMats}(\mathbb{F}, m)$.
2 **for** $i = 1$ **to** $t$ **do**
3 The parties compute and open $C_i \leftarrow \begin{cases} \mathsf{open}([R_i] \cdot [A_i] \cdot [R_{i+1}^{-1}]) & \text{if } i < t \\ \mathsf{open}([R_t] \cdot [A_t]) & \text{if } i = t. \end{cases}$
4 The parties compute $[B] \leftarrow [R_1^{-1}] \cdot (C_1 \cdot \cdots \cdot C_t)$.
5 **return** $[B]$

**Cost.** Similar to Section 3.3, offline complexity is 2 rounds and $2c_2tn^2$ communications, where $c_2$ is a constant defined in Section 3.2 and the online complexity is 3 rounds and $(3t-1)n^2$ communications.

## 3.5 Generating Invertible Matrix $[R]$ and $[\det(R)^{-1}]$

We present a procedure for jointly generating a uniformly random invertible matrix in shared form, together with a sharing of determinant of its inverse in Algorithm 5. In contrast to the protocol of Cramer and Damgård, which achieves the same goal [CD01, Section 6.1], our algorithm attains perfect privacy.

Each party $P_i$ locally samples an independent uniform random matrix $R_i \in \mathrm{GL}_n(\mathbb{F}_q)$ and secret-shares both $R_i$ and $\det(R_i)^{-1}$. The parties then compute $[R] = \prod_{i\in[N]}[R_i]$ and $[\det(R)^{-1}] = \prod_{i\in[N]}[\det(R_i)^{-1}]$ using Algorithm 3 and Algorithm 4, respectively.

Correctness follows from the multiplicativity of the determinant, namely $\det(R)^{-1} = \prod_{i\in[N]} \det(R_i)^{-1}$, and from closure of $\mathrm{GL}_n(\mathbb{F}_q)$ under multiplication. Moreover, since each $R_i$ is uniform over $\mathrm{GL}_n(\mathbb{F}_q)$, their product $R = \prod_{i\in[N]} R_i$ is also uniformly distributed over in $\mathrm{GL}_n(\mathbb{F}_q)$.

Although we describe the computation of $[\det(R)^{-1}]$ due to its use in Section 4, the procedure can be modified in a straightforward manner to output $[\det(R)]$ instead, by distributing $\det(R_i)$ in place of $\det(R_i)^{-1}$.

**Algorithm 5:** $\mathsf{GenInvMatDet}(\mathbb{F})$ - Generate invertible $[R]$ and $[\det(R)^{-1}]$

**Input** : None
**Output** : $([R], [\det(R)^{-1}])$ where $R \xleftarrow{\$} \mathrm{GL}_n(\mathbb{F})$

1 The parties uniformly sample their respective invertible matrices $R_i \xleftarrow{\$} \mathrm{GL}_n(\mathbb{F})$ for $i \in [N]$.
2 The parties locally computes $\det(R_i)$ for $i \in [N]$.
3 The parties distribute their matrices $[R_i] \leftarrow \mathsf{dist}(P_i, R_i)$ and their determinants $[\det(R_i)^{-1}] \leftarrow \mathsf{dist}(P_i, \det(R_i)^{-1})$ for $i \in [N]$.
4 The parties compute $[R] \leftarrow \mathsf{MultMats}(\mathbb{F}, [R_1], \ldots, [R_N])$.
5 The parties compute $[\det(R)^{-1}] \leftarrow \mathsf{MultScalars}(\mathbb{F}, [\det(R_1)^{-1}], \cdots, [\det(R_N)^{-1}])$.
6 **return** $[R], [\det(R)^{-1}]$

**Cost.** MultMats and MultScalars contain offline complexities of 2 rounds and $(2c_1+2c_2n^2)N$ communication in total. For online complexities, line 3 takes 1 round and $N(n^2+1)$ field element of communication, lines 4-5 take 3 additional rounds and $(3N-1)(n^2+1)$ communication, respectively. Overall, Algorithm 5 uses a total of 4 online rounds and $(4N-1)(n^2+1)$ communication.

### 3.6 Computing Matrix Powers

We describe the algorithm that computes cumulative powers of invertible matrix $[A]$ in Algorithm 6. We will focus on the powers of invertible matrices only, which is different from secure power matrix computation of [MV24] which utilizes double-size padded matrix instead of original matrix to hide the case that $\det(A)$ is zero.

**Algorithm 6:** $\mathsf{SecPower}(\mathbb{F}, A, m)$ - Compute cumulative powers of invertible matrix

**Input** : $[A] \in \mathrm{GL}_n(\mathbb{F}), m \in \mathbb{N}$
**Output**: $[A^2], [A^3], \cdots, [A^m]$

1 The parties generate $m$ pairs of random matrices $([R_i], [R_i^{-1}])_{i\in[m]}, \leftarrow \mathsf{GenInvMats}(\mathbb{F}_q, m, n)$.

2 The parties compute and open $N_i \leftarrow \begin{cases} \mathsf{open}([R_{i-1}] \cdot [A] \cdot [R_i^{-1}]) & \text{if } 2 \le i \le m \\ \mathsf{open}([A] \cdot [R_1^{-1}]) & \text{if } i = 1. \end{cases}$

3 The parties locally compute $P_j \leftarrow \prod_{i=1}^{j} N_i$ for $1 \le j \le m$.

4 The parties locally compute $[A^j] \leftarrow P_j[R_j]$ for $1 \le j \le m$.

5 **return** $[A^2], \cdots, [A^m]$

**Cost.** The offline complexity of this procedure is 2 rounds and $2c_2mn^2$ communication. This is from the line 1, which includes generating $m$ pairs of invertible matrix and its inverse. The total online complexity is 3 rounds and $(3m-1)n^2$ communication.

## 4 Lifting the Cramer-Damgård Protocol to Extension Fields

In this section, we present protocols with perfect privacy for secure matrix invertibility testing over small field order by lifting the Cramer-Damgård protocol to extension fields. Although the original work briefly notes this lifting idea [CD01], it does not provide a concrete analysis. We therefore give a detailed treatment, including a proof of perfect privacy as well as explicit bounds on round complexity and communication cost.

In the original Cramer-Damgård protocol, two aspects obstruct perfect privacy. First, the protocol generates a random invertible matrix $[R]$ together with $[\det(R)]$ via an LU-decomposition-based procedure and uses $R$ to mask the target matrix. However, the resulting distribution of $R$ is not uniform over $\mathrm{GL}_n(\mathbb{F})$. Second, the protocol fixes public points $z_0, z_1, \ldots, z_n \in \mathbb{F}$ and, for each $i$, considers the matrix $M_i := z_i I_n - A$. During the computation of $\det(M_i)$, the rank of $M_i$ becomes public, thereby leaking whether $z_i$ is an eigenvalue of $A$.

To address the first issue, we generate $([R], [\det(R)])$ such that $R$ is uniformly distributed in $\mathrm{GL}_n(\mathbb{F})$ using Algorithm 5. To address the second issue, we carry out the computation over the extension field $\mathbb{F}_{q^{n+1}}$. Let $A^{\mathsf{lift}}$ denote the natural embedding of $A$ into $\mathcal{M}_n(\mathbb{F}_{q^{n+1}})$. We sample $z_i \in \mathbb{F}_{q^{n+1}}$ such that its minimal polynomial over $\mathbb{F}_q$ has degree $n+1$. Then $z_i$ cannot be a root of $\chi_{A^{\mathsf{lift}}}(X)$, since any eigenvalue of $A^{\mathsf{lift}}$ is algebraic over $\mathbb{F}_q$ of degree at most $n$. Equivalently, for every $i$, the matrix $M_i := z_i I_n - A^{\mathsf{lift}}$ is nonsingular over $\mathbb{F}_{q^{n+1}}$. Consequently, the publicly revealed ranks are always equal to $n$ and therefore do not leak information about $A$.

### 4.1 Protocol Description

In the protocol, $z_i \in \mathbb{F}_{q^{n+1}}$ are public constants whose minimal polynomials over $\mathbb{F}_q$ have degree $n+1$.

**Algorithm 7:** $\mathsf{isInverseCD}(A)$ - Invertibility test by lifting the Cramer-Damgård protocol over extension fields

**Input** : $A \in \mathcal{M}_n(\mathbb{F}_q)$
**Output** : 0 if $\det(A) = 0$, 1 else

1 The parties generate $([R_i], [\det(R_i)^{-1}]) \leftarrow \mathsf{GenInvMatDet}(\mathbb{F}_{q^{n+1}})$ for $i \in [n+1]$.
2 The parties generate a nonzero random element $([r], [r^{-1}]) \leftarrow \mathsf{GenNonzeroElems}(\mathbb{F}_{q^{n+1}}, 1)$.
3 The parties compute $[r_i] = [r] \cdot [\det(R_i)^{-1}]$ for $i \in [n+1]$.
4 The parties lift $[A^{\mathsf{lift}}] \leftarrow \mathsf{lift}([A], \mathbb{F}_{q^{n+1}})$.
5 The parties set $[C_i] \leftarrow [A^{\mathsf{lift}}] - z_i \cdot I$.
6 The parties compute and open $[B_i] \leftarrow [C_i] \cdot [R_i]$.
7 The parties locally compute $\det(B_i)$.
8 The parties locally compute $[r \cdot \chi_{A^{\mathsf{lift}}}(0)] = \sum_{i \in [n+1]} \lambda_i \cdot \det(B_i) \cdot r_i$
9 The parties open $r \cdot \chi_{A^{\mathsf{lift}}}(0) \leftarrow \mathsf{open}([r \cdot \chi_{A^{\mathsf{lift}}}(0)])$.
10 **return** $\begin{cases} 0 & \text{if } r \cdot \chi_{A^{\mathsf{lift}}}(0) = 0 \\ 1 & \text{otherwise.} \end{cases}$

### 4.2 Cost

We examine the cost line by line.

- **Lines 1-2**. These lines require 6 offline rounds and $(4N-1)(n^2+1)(n+1)^2 + (2c_1+2c_2n^2)(n+1)^2N$ communication. Although Algorithm 5 includes both offline and online components, in our setting these computations can be performed prior to receiving $A$ and are therefore classified as offline. There are $(n+1)$ calls of $\mathsf{GenInvMatDet}(\mathbb{F}_{q^{n+1}})$ and each requires $(4N-1)(n^2+1)(n+1)+(2c_1+2c_2n^2)(n+1)N$ communication. Moreover, by sampling one additional random element during the execution of $\mathsf{GenInvMatDet}$, Lines 1 and 2 can be executed in parallel.
- **Line 3.** This line requires 1 offline round and $(n+1)^2$ communication.
- **Line 6.** This line requires 2 online rounds and $2n^2(n+1)$ communication.
- **Line 9.** requires 1 online round and $(n+1)$ communication.

Overall, the protocol uses 7 offline rounds with communication complexity $O(N \cdot n^4 + n^5)$ and 3 online rounds with communication complexity $O(n^3)$.

## 5 Applying the Samuelson-Berkowitz Protocol

For small characteristic field ($p \leq n$), we cannot directly apply Maire and Vergnaud's protocol using Lemma 1 since at least one of diagonal entry of matrix in left-hand side become zero thus singular. They stated that Schönhage's protocol [Sch93], which use Lemma 1 over extended domain $\mathbb{F}_q[y]/(y^{n+1})$ with appropriate companion matrix, can cover this special case. However, we analyzed that multi-party version of Schönhage's protocol takes 2 offline rounds with $O(n^3)$ and 11 online rounds with $O(n^4)$ communication, which takes more online rounds with respect to our second protocol described below. The detailed analysis is provided in Appendix A.

Instead of using both Lemma 1 and Schönhage's protocol, we propose our second invertibility test protocol utilizing Samuelson-Berkowitz algorithm [Ber84] in MPC manner. Given matrix $A \in \mathcal{M}_n(\mathbb{F}_q)$, we first define its $k$-th diagonal entries $a_{k,k}$, submatrices $A_k, M_k, R_k, S_k$ for $k \in [n]$ as follows.

$$A_k := \left[\begin{array}{c|c} a_{k,k} & R_k \\ \hline S_k & M_k \end{array}\right]$$

Using above submatrices, we can construct $(n-k+2)\times(n-k+1)$ Toeplitz matrix $C_k$ for $k\in[n]$ described below. The Samuelson-Berkowitz algorithm states that the vector $v = T_0T_1\cdots T_{n-1}$ contains coefficients of characteristic polynomial $\chi_A(x)$, so that we can directly test matrix invertibility using $v$.

$$C_k := \begin{bmatrix} a_{0,0} & & & & & & \\ a_{k,k} & -1 & & & & & \\ R_kM_k^0S_k & a_{k,k} & -1 & & & & \\ R_kM_k^1S_k & a_{k,k} & -1 & & & & \\ R_kM_k^2S_k & R_kM_k^1S_k & a_{k,k} & -1 & & & \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \\ R_kM_k^{n-k-2}S_k & \cdots & \cdots & \cdots & R_kM_k^0S_k & a_{k,k} & -1 \\ R_kM_k^{n-k-1}S_k & R_kM_k^{n-k-2}S_k & \cdots & \cdots & \cdots & R_kM_k^0S_k & a_{k,k} \end{bmatrix}$$

## 5.1 Protocol Description

Our second protocol using multi-party version of Samuelson-Berkowitz algorithm is stated in Algorithm 8. Regarding the information leakage, Algorithm 8 only opens full-rank matrices $[U_k]_{k\in[n]}$ and masked determinant $p_n$. Therefore, this protocol meets perfect privacy.

**Algorithm 8:** $\mathsf{isInverseSB}(A)$ - Invertibility test using Samuelson-Berkowitz algorithm

**Input** : $A\in\mathcal{M}_n(\mathbb{F}_q)$
**Output** : 0 if $\det(A)=0$, 1 else

1 The parties obtain $[M_k],[R_k]$, and $[S_k]$ for $k\in[n]$ as described above.
2 The parties generate $([T_k],[T_k^{-1}]) \leftarrow \mathsf{GenInvMats}(\mathbb{F}_q, 1, n-k+2)$ for $k\in[n]$.
3 The parties generate random field element $([r],[r^{-1}]) \leftarrow \mathsf{GenNonzeroElems}(\mathbb{F}_q,1)$.
4 The parties compute $[T_1'] = [r]\cdot[T_1]$.
5 **for** $k\in[n]$ in parallel **do**
6  The parties compute $[M_k^2],\cdots,[M_k^{n-k-1}] \leftarrow \mathsf{SecPower}(\mathbb{F}_q, M_k, n-k-1)$.
7  The parties obtain $[C_k]$ by computing $[R_k][M_k^0][S_k],\cdots,[R_k][M_k^{n-k-1}][S_k]$.
8  **if** $k\neq n$ **then**
9  The parties compute and open $U_k \leftarrow \mathsf{open}([T_k^{-1}][C_k][T_{k+1}])$.
10  **else**
11  The parties compute and open $U_n \leftarrow \mathsf{open}([T_n^{-1}][C_n])$.
12 The parties locally compute $\left[\begin{pmatrix} p_0 & p_1 & \cdots & p_n\end{pmatrix}^\top\right] \leftarrow [T_1']\cdot U_1U_2\cdots U_n$ and open $p_n \leftarrow \mathsf{open}([p_n])$.
13 **return** $\begin{cases} 0 & \text{if } p_n = 0 \\ 1 & \text{otherwise.}\end{cases}$

## 5.2 Cost

We examine the cost line by line.

1. **Line 2.** This line requires $n$ invocation of $\mathsf{GenInvMats}$, which takes 2 offline round and $2c_2^*\{\sum_{k\in[n]}(n-k+2)^2\}$ communication. Since each $\mathsf{GenInvMats}$ instance has different constant $c_2$, we choose $c_2^*$ as maximum of them.
2. **Line 3.** This line requires 2 offline round and $2c_1$ communication.
3. **Line 4.** This line requires 1 offline round and $(n+1)^2$ communication.

4. **Line 6.** This line requires $\mathsf{SecPower}(\mathbb{F}_q, M_k, n-k-1)$ where $[M_k]$ is an $(n-k)\times(n-k)$, which takes 2 offline round with $2c_2(n-k-1)(n-k)^2$ communication and 3 online rounds with $(3n-3k-4)(n-k)^2$ communication for each $k$.
5. **Line 7.** This line requires $n-k$ multiplications of matrices of size $(n-k)\times(n-k)$ and vectors of length $n-k$ and $n-k$ inner products of two vectors of length $n-k$; thus it takes 2 online rounds and $(n-k)(n-k+1)$ communication for each $k$.
6. **Lines 8-11.** These lines require 3 online rounds and $3(n-k+2)(n-k+1)$ communication if $k \neq n$ or 4 communication if $k = n$ including matrix-vector multiplication and opening.
7. **Line 12.** This line requires 1 online round and 1 communication for opening.

For line 5-11, total communication will be summed over $k \in [n]$ since computation is done in parallel for each $k$. Lines 2, 3, and 6 allow for concurrent execution, but Line 4 must strictly follow the completion of Lines 2 and 3. As a result, the overall complexity of this protocol is 3 offline rounds with communication complexity $O(n^3)$ and 9 online rounds with communication complexity $O(n^4)$.

# 6 Conclusion

In this paper, we revisited the problem of securely testing matrix invertibility over fields of small order or characteristics in the MPC setting, where classical constant-round protocols may lose their appeal due to non-negligible failure probability and privacy leakage. Working in the Arithmetic Black-Box (ABB) model, we presented two complementary protocol families tailored to the fields of small order or characteristic.

We emphasize that our protocols achieve perfect privacy and constant round over fields of small order or characteristic, where standard constant-round techniques typically incur non-negligible failure probability or leak algebraic information.

Several directions remain for future work. On the practical side, it would be valuable to integrate these protocols into concrete MPC frameworks (e.g., SPDZ-style or replicated secret sharing) and evaluate end-to-end latency under realistic network conditions. On the algorithmic side, further reducing the extension-field overhead and tightening the constants in preprocessing are promising avenues.

# References

[AFL+16] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 805–817. ACM Press, October 2016. doi:10.1145/2976749.2978331.

[BCC+23] Ward Beullens, Fabio Campos, Sofía Celi, Basil Hess, and Matthias J. Kannwischer. MAYO. Technical report, National Institute of Standards and Technology, 2023. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[BCD+24] Ward Beullens, Ming-Shing Chen, Jintai Ding, Boru Gong, Matthias J. Kannwischer, Jacques Patarin, Bo-Yuan Peng, Dieter Schmidt, Cheng-Jhih Shih, Chengdong Tao, and Bo-Yin Yang. UOV — Unbalanced Oil and Vinegar. Technical report, National Institute of Standards and Technology, 2024. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

[Ber84] Stuart J Berkowitz. On computing the determinant in small parallel time using a small number of processors. *Information processing letters*, 18(3):147–150, 1984.

[BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988. doi:10.1145/62212.62213.

[BIB89] Judit Bar-Ilan and Donald Beaver. Non-cryptographic fault-tolerant computing in constant number of rounds of interaction. In Piotr Rudnicki, editor, *8th ACM PODC*, pages 201–209. ACM, August 1989. doi:10.1145/72981.72995.

[CD01] Ronald Cramer and Ivan Damgård. Secure distributed linear algebra in a constant number of rounds. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 119–136. Springer, Berlin, Heidelberg, August 2001. doi:10.1007/3-540-44647-8_7.

[CDI05] Ronald Cramer, Ivan Damgård, and Yuval Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 342–362. Springer, Berlin, Heidelberg, February 2005. doi:10.1007/978-3-540-30576-7_19.

[CEN25] Sofía Celi, Daniel Escudero, and Guilhem Niot. Share the MAYO: Thresholdizing MAYO. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, *Post-Quantum Cryptography - 16th International Workshop, PQCrypto 2025, Part I*, pages 165–198. Springer, Cham, April 2025. doi:10.1007/978-3-031-86599-2_6.

[CKP07] Ronald Cramer, Eike Kiltz, and Carles Padró. A note on secure computation of the Moore-Penrose pseudoinverse and its application to secure linear algebra. In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 613–630. Springer, Berlin, Heidelberg, August 2007. doi:10.1007/978-3-540-74143-5_34.

[CS19] Daniele Cozzo and Nigel P. Smart. Sharing the LUOV: Threshold post-quantum signatures. In Martin Albrecht, editor, *17th IMA International Conference on Cryptography and Coding*, volume 11929 of *LNCS*, pages 128–153. Springer, Cham, December 2019. doi:10.1007/978-3-030-35199-1_7.

[DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Berlin, Heidelberg, August 2012. doi:10.1007/978-3-642-32009-5_38.

[MV24] Jules Maire and Damien Vergnaud. Secure multi-party linear algebra with perfect correctness. *CiC*, 1(1):29, 2024. doi:10.62056/avzojbkrz.

[MW08] Payman Mohassel and Enav Weinreb. Efficient secure linear algebra in the presence of covert or computationally unbounded adversaries. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 481–496. Springer, Berlin, Heidelberg, August 2008. doi:10.1007/978-3-540-85174-5_27.

[Sch93] Arnold Schönhage. Fast parallel computation of characteristic polynomials by leverrier's power sum method adapted to fields of finite characteristic. In *International Colloquium on Automata, Languages, and Programming*, pages 410–417. Springer, 1993.

# A Analysis of [MV24] Adapted with the Schönhage Algorithm

In this section, we analyze the detailed round and communication complexity of the invertibility testing algorithm from [MV24] adapted with the the Schönhage algorithm [Sch93]. Although Maire and Vergnaud suggested this substitution to handle small characteristic fields, a concrete complexity analysis for this modified construction was omitted.

For formal definitions of constants $m, r, l, \mu, \rho$, locally computable matrices $[D], [Q], [u], [p']$ and companion matrix $C$ used in Algorithm 9, we direct the reader to [Sch93].

**Algorithm 9:** Modified invertibility testing protocol from [MV24] adapted with the Schönhage algorithm [Sch93] for small characteristic fields

**Input** : $A \in \mathcal{M}_n(\mathbb{F}_q)$
**Output** : 0 if $\det(A) = 0$, 1 else

1 Parties appropriately construct zero-padded $n' \times n'$ matrix $[A'] = \mathsf{Pad}([A])$ such that its size $n' \equiv 1 \mod q$.
2 Parties locally computes $m, r, l, \mu, \rho$ and companion matrix $C$ of polynomial $\hat{f}(t)$ given in the Proposition 3 of [Sch93] and set $[B] := C - yC[A']$.
▷ Perform further steps in $\mathbb{F}'_q = \mathbb{F}_q[y]/y^{n'+1}$.
3 Parties generate 2 pairs of random matrices $([R_1], [R_1^{-1}]) \leftarrow \mathsf{GenInvMats}(\mathbb{F}'_q, 1, n'), ([R_2], [R_2^{-1}]), \leftarrow \mathsf{GenInvMats}(\mathbb{F}'_q, 1, \mu)$.
4 Parties invoke $\mathsf{SecPower}(\mathbb{F}'_q, B, l)$ and get $[s_j] := tr([B^j])$.
5 Parties solve $[v] = [(I - D)^{-1}][u]$ using $([R_1], [R_1^{-1}])$ where $v \in (\mathbb{F}'_q)^{n'}$. Here, $[D]$ and $[u]$ are locally computable.
6 Parties solve $[([a_q], [a_{2q}], ..., [a_{\mu q}])] = [a'] := -[Q^{-1}][p']$ using $([R_2], [R_2^{-1}])$. Here, $[Q]$ and $[p']$ are locally computable.
7 Parties compute $\det(I - yA) = a_{n'} := \sum_{i+j=\mu}[a_{iq}] \cdot [p_{1,j}]$ and open $[p_{n'}]$, which is a coefficient of $y^{n'}$ in $a_{n'}$.
8 **return** $\begin{cases} 0 & \text{if } p_{n'} = 0 \\ 1 & \text{otherwise.} \end{cases}$

**Cost.** We examine cost line by line. After line 2, all arithmetic operations are done in extended field $\mathbb{F}'_q$, hence communicated field elements are multiplied by $n'$.

1. **Line 3.** This line requires 2 offline rounds with $2c_1 n'(n'^2 + \mu^2)$ communications for Shamir's secret sharing.
2. **Line 4.** This line requires 2 offline rounds with $4c_2 l^2 n'$ and 3 online rounds with $(3l - 1)n'^3$ communications.
3. **Line 5.** This line requires 3 online rounds and $2(n')^3 + (n')^2$ communications. For computing $[(I - D)^{-1}]$, we use one–sided masking method with $([R_1], [R_1^{-1}])$.
4. **Line 6.** This line requires 3 online rounds and $\mu(2\mu + 1)n'$ communications. For computing $[Q^{-1}]$, we use one–sided masking method with $([R_2], [R_2^{-1}])$.
5. **Line 7.** This line requires 2 online rounds and $(\mu + 1)n'$ communications.

For the field of characteristic 2, we can assume that $l \approx 2n, n' = n + 1$ and $\mu \approx n/2$. Line 3 and 4 allow for concurrent execution. As a result, this algorithm takes 2 offline rounds with $O(n^3)$ and 11 online rounds with $O(n^4)$ communications.